

Lab Manual

AIC341 - Introduction to Computer Vision



(1st Draft Version)

CUI

**Department of Computer Science
Islamabad Campus**

Lab Contents:

The topics include: Basics of Python for AI , Different Searching techniques categorized into heuristical and blind , Analytical problems for testing AI approaches like N-Queens, Introduction to Genetic Algorithms , Introduction to Game Theory including min-max and alpha-beta pruning , and developing game modules like TIC-TAC-TOE , GO and Cheaquers

Student Outcomes (SO)

S.#	Description
2	Identify, formulate, research literature, and solve complex computing problems reaching substantiated conclusions using fundamental principles of mathematics, computing sciences, and relevant domain disciplines
3	Design and evaluate solutions for complex computing problems, and design and evaluate systems, components, or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations
4	Create, select, adapt and apply appropriate techniques, resources, and modern computing tools to complex computing activities, with an understanding of the limitations
5	Function effectively as an individual and as a member or leader in diverse teams and in multi-disciplinary settings.

Intended Learning Outcomes

Sr.#	Description	Blooms Taxonomy Learning Level	SO
CLO -5	Implement various computer vision techniques and utilize OpenCV to address image processing challenges.	<i>Applying</i>	2-4
CLO-6	Develop a medium-sized project incorporating OpenCV for computer vision tasks within a team environment.	<i>Applying</i>	2-5

Lab Assessment Policy

The lab work done by the student is evaluated using Psycho-motor rubrics defined by the course instructor, viva-voce, project work/performance. Marks distribution is as follows:

Assignments	Lab Mid Term Exam	Lab Terminal Exam	Total
25	25	50	100

Note: Midterm and Final term exams must be computer based.

List of Labs

Lab #	Main Topic	Page #
Lab 01	Introduction to Computer Vision with OpenCV	4
Lab 02	Drawing Functions	8
Lab 03	Video Processing with OpenCV	14
Lab 04	OpenCV Edge Detection and Blurring	18
Lab 05	Morphological Operations OpenCV	22
Lab 06	Play a video in reverse mode Using OpenCV	24
Lab 07	Track objects with Camshift using OpenCV	25
Mid Term Exam		
Lab 08	Feature Extraction with Traditional Approaches	26
Lab 9	OCR	27
Lab 10	Mini Project 1- Identify Contours by Shape	28
Lab 11	Mini Project 2 - Blob Detection	30
Lab 12	Counting Circles and Ellipses	32
Lab 13	Face Recognition – Unlock Your Computer with Your Face!	38
Lab 14	Training YOLOv8 on a Custom Dataset	40
Final Term Exam		
	*Can be added as per Committee Recommendation	

Lab 01

Introduction to Computer Vision with OpenCV

Objective:

This lab serves as an introduction to Computer Vision using the OpenCV library in Python. Computer Vision is a field that empowers machines to interpret and understand visual information, and OpenCV is a powerful tool for implementing computer vision applications.

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Import and utilize OpenCV in Python
- Read and display images using OpenCV
- Perform basic image operations, such as resizing, rotating, and flipping
- Apply image filtering techniques, including blurring and grayscale conversion
- Draw on images to annotate and highlight features
- Save processed images to the local filesystem

Instructor Note:

Before starting this lab, it is recommended to have a basic understanding of Python programming. If needed, refer to the provided resources or any Python programming guide. Familiarity with basic image processing concepts will also be beneficial. As a pre-lab activity, review relevant sections on image processing and OpenCV documentation to prepare for the hands-on exercises.

1) Solved Lab Activities (Allocated Time 1 Hour)

Pre Lab:

Please read the solution below for understanding before starting the lab

<https://softwaremill.com/what-is-opencv-used-for/>

Activity 1:

Import and Utilize OpenCV in Python

Solution:

1. **Open Python IDE:**
 - a. Launch your Python IDE (e.g., IDLE or Jupyter Notebook).
2. **Import OpenCV:**
 - a. Type **import cv2** and press Enter.
 - b. Ensure there are no errors, indicating successful import.

Activity 2:

Read and Display Images using OpenCV

Solution:

1. **Read an Image:**
 - a. Type `image = cv2.imread('path_to_image.jpg')` and press Enter.
 - b. Replace 'path_to_image.jpg' with the actual path of an image file.
2. **Display the Image:**
 - a. Type `cv2.imshow('Image', image)` and press Enter.
 - b. Add `cv2.waitKey(0)` and `cv2.destroyAllWindows()` to keep the window open until a key is pressed.

Activity 3: Perform Basic Image Operations

Solution:

1. **Resize the Image:**
 - a. Type `resized_image = cv2.resize(image, (new_width, new_height))` and press Enter.
2. **Rotate the Image:**
 - a. Type `rotated_image = cv2.rotate(image, cv2.ROTATE_90_CLOCKWISE)` and press Enter.
3. **Flip the Image Horizontally:**
 - a. Type `flipped_image = cv2.flip(image, 1)` and press Enter.

Activity 4: Apply Image Filtering Techniques

Solution:

1. Apply Gaussian Blur:
 - a. Type `blurred_image = cv2.GaussianBlur(image, (kernel_size, kernel_size), 0)` and press Enter.
2. Convert Image to Grayscale:
 - a. Type `gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)` and press Enter.

Activity 5: Draw on Images to Annotate and Highlight Features

Solution:

1. Draw a Rectangle:
 - a. Type `cv2.rectangle(image, (start_x, start_y), (end_x, end_y), (0, 255, 0), 2)` and press Enter.
2. Add Text:
 - a. Type `cv2.putText(image, 'Hello, OpenCV!', (start_x, start_y), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 0, 0), 2)` and press Enter.

Activity 6: Save Processed Images to the Local Filesystem

Solution:

1. Save Image:
 - a. Type `cv2.imwrite('output_image.jpg', image)` and press Enter.
 - b. Check your directory for the newly saved image.

2) Graded Lab Tasks (Allotted Time 1.5 Hours)

Pre-Lab:

Before performing the lab, please read the material below.

<https://medium.com/nerd-for-tech/using-opencv-with-python-to-perform-basic-operations-on-images-bb396153ae2a>

Lab Task 1:

Basic Image Operations

Objective:

Perform basic image operations like resizing, rotating, and flipping.

Instructions:

- Read any open-source image.
- Resize the image to half its original dimensions.
- Rotate the resized image 180 degrees clockwise.
- Flip the rotated image vertically.

Lab Task 2:

Image Filtering Techniques

Objective:

Apply image filtering techniques, including blurring and grayscale conversion.

Instructions:

- Take the image from Exercise 2.
- Blur the image using a Gaussian blur with a kernel size of 5x5.
- Convert the blurred image to grayscale.
- Saved the image

Lab 02

Drawing Functions

Objective:

Drawing functions in OpenCV facilitate the manipulation of images and matrices, providing the ability to create and visualize various shapes, lines, and text. These functions are designed to work with matrices or images of arbitrary depth, allowing for a wide range of applications in computer vision and image processing.

Solved Lab Activities (Allocated Time 1 Hour)

Pre Lab:

Before performing the lab, please read the material below.

https://docs.opencv.org/2.4/modules/core/doc/drawing_functions.html#circle

<https://www.geeksforgeeks.org/python-opencv-cv2-line-method/?ref=lbp>

Activity 1:

Drawing Circles

- **Load an Image:**
 - Choose an image (e.g., 'sample_image.jpg').
- **Read and Display the Image:**

```
import cv2

image = cv2.imread('sample_image.jpg')
cv2.imshow('Original Image', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

○

- **Draw a Circle:**

```
import cv2

# Coordinates and parameters for the circle
center_coordinates = (150, 100)
radius = 20
color = (0, 255, 0) # Green color in BGR
thickness = 2

# Draw the circle on the image
image_with_circle = cv2.circle(image, center_coordinates, radius, color, thickness)

# Display the image with the circle
cv2.imshow('Image with Circle', image_with_circle)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

○

Activity 2:

Drawing Lines

- **Load an Image:**
 - Choose an image (e.g., 'sample_image.jpg').
- **Read and Display the Image:**

```
import cv2

image = cv2.imread('sample_image.jpg')
cv2.imshow('Original Image', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

○

- Draw a Line:

```
import cv2

# Coordinates and parameters for the line
start_point = (50, 50)
end_point = (200, 100)
color = (0, 255, 0) # Green color in BGR
thickness = 2

# Draw the line on the image
image_with_line = cv2.line(image, start_point, end_point, color, thickness)

# Display the image with the line
cv2.imshow('Image with Line', image_with_line)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

○

1) Graded Lab Tasks (Allotted Time 1.5 Hours)

Task 1: Drawing Ellipses

- **Objective:**
 - Draw ellipses on an image using OpenCV.
- **Instructions:**
 - Load an image of your choice.
 - Draw at least three ellipses with different parameters (varying center, axes lengths, angles, and colors).
 - Display the image with the drawn ellipses.

Task 2: Converting Ellipses to Polygons

- **Objective:**
 - Utilize the **ellipse2Poly** function to convert ellipses to polygons.
- **Instructions:**
 - Choose an ellipse from Task 1.
 - Use **ellipse2Poly** to convert the ellipse to a polygon.
 - Display the image with the original ellipse and the polygonized version.

Task 3: Filling Convex Polygons

- **Objective:**
 - Implement the **fillConvexPoly** function to fill convex polygons.
- **Instructions:**
 - Select a convex polygon (you can use the polygonized ellipse from Task 2).
 - Fill the convex polygon with a color of your choice.
 - Display the image with the filled convex polygon.

Task 4: Drawing Rectangles

- **Objective:**
 - Draw rectangles on an image.
- **Instructions:**
 - Choose an image.
 - Draw at least two rectangles with different parameters (varying start and end points, colors, and thickness).
 - Display the image with the drawn rectangles.

Task 5: Drawing Lines and Arrowed Lines

- **Objective:**
 - Implement the **line** and **arrowedLine** functions to draw lines on an image.
- **Instructions:**
 - Select two points and draw a line connecting them.
 - Draw an arrowed line starting from one point and ending at another.
 - Display the image with the drawn lines.

Task 6: Drawing Polylines

- **Objective:**
 - Utilize the **polylines** function to draw multiple connected line segments.
- **Instructions:**
 - Define a set of points to create a polyline (a shape with multiple connected line segments).
 - Draw the polyline on the image.
 - Display the image with the drawn polyline.

Task 7: Putting Text on Images

- **Objective:**
 - Use the **putText** function to write text on an image.
- **Instructions:**
 - Choose an image.
 - Add text with different font sizes, colors, and positions.
 - Display the image with the added text.

Task 8: Additional Challenges

- **Objective:**
 - Explore additional drawing functions (e.g., **getTextSize**, **InitFont**, **LineIterator**).
- **Instructions:**
 - Implement any two additional drawing functions.
 - Experiment with various parameters and observe the effects.
 - Document your findings and display the image with the applied functions.

Submission:

- Combine all the implemented tasks into a single Python script.
- Add comments and documentation explaining each task.
- Display the resulting images for each task.
- Submit the Python script and any additional documentation.

Lab 03

Video Processing with OpenCV

1) Solved Lab Activities (Allocated Time 1 Hour)

Pre Lab:

Before performing the lab, please read the material below.

<https://www.geeksforgeeks.org/python-create-video-using-multiple-images-using-opencv/?ref=lbp>

Activity 1:

Play a Video using OpenCV

- **Load a Video File:**

- Choose a video file (e.g., 'sample_video.mp4').
- Load the video using OpenCV.

```
import cv2

# Video file path
video_path = 'sample_video.mp4'

# Open the video file
cap = cv2.VideoCapture(video_path)
```

-
- **Play the Video:**

- Display each frame of the video.
- Add a delay between frames.

```
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    # Display the frame
    cv2.imshow('Video Playback', frame)

    # Exit on 'q' key press
    if cv2.waitKey(25) & 0xFF == ord('q'):
        break

# Release the video capture object
cap.release()
cv2.destroyAllWindows()
```

○

Activity 2:

Create Video using Multiple Images

- **Load Images:**
 - Choose a set of images (e.g., 'image1.jpg', 'image2.jpg', ...).
 - Read each image using OpenCV.
- **Create a Video:**
 - Combine the images to create a video.
 - Set video parameters such as frame width, height, and frames per second (fps).

2) Graded Lab Tasks (Allotted Time 1.5 Hour)

Pre

Please read the following material before starting the lab.

<https://www.geeksforgeeks.org/python-process-images-of-a-video-using-opencv/?ref=lbp>

<https://www.geeksforgeeks.org/python-opencv-capture-video-from-camera/?ref=lbp>

Lab:**Lab Task 1:****Lab Task 1: Processing Images of a Video**

- **Objective:**
 - Implement image processing techniques on frames extracted from a video using OpenCV.
- **Instructions:**
 - Choose a video file (e.g., 'sample_video.mp4').
 - Load the video and process each frame using OpenCV.
 - Apply at least three different image processing techniques (e.g., resizing, blurring, color manipulation) to the frames.
 - Display the original and processed frames side by side for visual comparison.

Lab Task 2: Writing to Video with OpenCV

- **Objective:**
 - Create a new video by writing processed frames to a video file using OpenCV.
- **Instructions:**
 - Extend the code from Lab Task 1.
 - After processing each frame, write the processed frame to a new video file.
 - Set video parameters such as frame width, height, and frames per second (fps).
 - Display a message indicating the progress of video creation.

Lab Task 3: Writing Text on Video

- **Objective:**
 - Enhance the created video by adding text on each frame.
- **Instructions:**

-
- Extend the code from Lab Task 2.
 - Write dynamic text on each frame, indicating the frame number or any relevant information.
 - Choose a distinctive font, size, and color for the text.
 - Display a message indicating the completion of the video creation process.

Submission:

- Combine all the implemented tasks into a single Python script.
- Add comments and documentation explaining each task.
- Display the original frames, processed frames, and the final video.
- Submit the Python script and any additional documentation.

Helping materials:

<https://www.geeksforgeeks.org/python-process-images-of-a-video-using-opencv/?ref=lbp>

<https://www.geeksforgeeks.org/python-opencv-capture-video-from-camera/?ref=lbp>

Lab 04

OpenCV Edge Detection and Blurring

1) Solved Lab Activities (Allocated Time 1 Hour)

Pre Lab:

Please read the following material before starting the lab.

<https://learnopencv.com/edge-detection-using-opencv/>

Activity 1:

Edge Detection using OpenCV

- **Load and Display an Image:**
 - Choose an image (e.g., 'sample_image.jpg').
 - Load and display the original image using OpenCV.

```
import cv2
import numpy as np

# Load the image
image = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)

# Display the original image
cv2.imshow('Original Image', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

-
- **Apply Edge Detection:**
 - Utilize an edge detection algorithm, such as the Canny edge detector.

- Display the resulting image after edge detection.

```
# Apply Canny edge detection
edges = cv2.Canny(image, 50, 150)

# Display the image with edges
cv2.imshow('Edge Detection', edges)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

○

Activity 2:

Image Blurring using OpenCV

- **Load and Display an Image:**

- Choose an image (e.g., 'sample_image.jpg').
- Load and display the original image using OpenCV.

```
import cv2
import numpy as np

# Load the image
image = cv2.imread('sample_image.jpg')

# Display the original image
cv2.imshow('Original Image', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

○

- **Apply Blurring Techniques:**

- Use various blurring techniques like Gaussian blur and Median blur.
- Display the resulting images after applying each blurring technique.

```
# Apply Gaussian blur
blurred_gaussian = cv2.GaussianBlur(image, (5, 5), 0)
cv2.imshow('Gaussian Blur', blurred_gaussian)
cv2.waitKey(0)

# Apply Median blur
blurred_median = cv2.medianBlur(image, 5)
cv2.imshow('Median Blur', blurred_median)
cv2.waitKey(0)

cv2.destroyAllWindows()
```

○

1) Graded Lab Tasks (Allotted Time 1.5 Hour)

Pre

Lab:

Please read the following material before starting the lab.

https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_filtering/py_filtering.html

Lab Task 1:

Advanced Edge Detection Techniques

- **Objective:**
 - Explore and implement advanced edge detection techniques using OpenCV.
- **Instructions:**
 - Choose an image for testing.
 - Implement at least two additional edge detection techniques (e.g., Sobel, Scharr, Laplacian) using OpenCV.
 - Display the resulting images after applying each edge detection technique.
 - Provide a comparative analysis of the different edge detection methods used.

Helping materials:

<https://learnopencv.com/edge-detection-using-opencv/>

Lab Task 2:

Custom Blurring Filters

- **Objective:**
 - Experiment with custom blurring filters and their effects on an image.
- **Instructions:**
 - Select an image for testing.
 - Implement a custom blurring filter using convolution operations (you can use `cv2.filter2D`).
 - Display the image after applying the custom blurring filter.
 - Experiment with different kernel sizes and shapes.
 - Compare the results with the standard blurring techniques used in the previous lab activity.

Helping

material:

https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_filtering/py_filtering.html

Lab 05

Morphological Operations

2) Graded Lab Tasks (Allotted Time 1.5 Hour)

Pre Lab:

Please read the following material before starting the lab.

<https://www.geeksforgeeks.org/python-opencv-morphological-operations/?ref=lbp>

Lab Task 1: Erosion and Dilation

- **Objective:**
 - Implement and understand the effects of erosion and dilation morphological operations using OpenCV.
- **Instructions:**
 - Choose an image for testing.
 - Implement erosion and dilation on the image with varying kernel sizes and iterations.
 - Display the original image along with the results after erosion and dilation.
 - Provide a brief explanation of the impact of different kernel sizes and iteration values.

Lab Task 2: Opening and Closing

- **Objective:**
 - Apply opening and closing morphological operations and analyze their effects on an image.
- **Instructions:**
 - Choose a different image for testing.
 - Implement opening and closing on the image using appropriate structuring elements.
 - Display the original image and the results after opening and closing.
 - Discuss how opening and closing operations influence different types of image structures.

Lab Task 3: Morphological Gradient

- **Objective:**
 - Implement the morphological gradient operation and understand its significance.
- **Instructions:**
 - Select a new image for testing.
 - Apply the morphological gradient operation and display the resulting image.
 - Discuss how the morphological gradient highlights edges and boundaries in the image.
 - Experiment with different structuring elements and observe the effects.

Lab Task 4: Top Hat

- **Objective:**
 - Implement the top hat morphological operation and analyze its impact.

-
- **Instructions:**
 - Choose another image for testing.
 - Apply the top hat operation and display the original image along with the result.
 - Explain how the top hat operation helps in revealing details in an image.

Lab Task 5: Black Hat

- **Objective:**
 - Implement the black hat morphological operation and evaluate its effects.
- **Instructions:**
 - Select a different image for testing.
 - Apply the black hat operation and display the original image and the result.
 - Discuss the application scenarios where the black hat operation can be useful.

Submission:

- Combine all the implemented tasks into a single Python script.
- Add comments and documentation explaining each task.
- Display the resulting images for each task.
- Provide a summary of your observations and findings for each morphological operation.

Helping

<https://www.geeksforgeeks.org/python-opencv-morphological-operations/?ref=lbp>

materials:

Lab 06

Play a video in reverse mode

Pre lab:

Before starting the Lab, please read the following materials first.

<https://www.geeksforgeeks.org/python-program-extract-frames-using-opencv/>

<https://www.futurelearn.com/info/courses/introduction-to-image-analysis-for-plant-phenotyping/0/steps/305359>

Lab Task: Play a Video in Reverse Mode using OpenCV

- **Objective:**
 - Develop a Python script to play a video in reverse mode using OpenCV.
- **Instructions:**
 - Choose a video file for testing (e.g., 'sample_video.mp4').
 - Implement a Python script that loads the video and plays it in reverse.
 - Display the original and reversed video side by side for comparison.
 - Ensure smooth and responsive video playback.

Lab 07

Object Tracking with Camshift using OpenCV

Pre Lab:

Before starting the project, please read the following materials first.

<https://data-flair.training/blogs/camshift-object-tracking-opencv/>

1) Graded Lab Tasks (Allotted Time 1.5 Hour)

Lab Task: Object Tracking with Camshift using OpenCV

- **Objective:**
 - Implement a Python script for object tracking using the Camshift algorithm in OpenCV.
- **Instructions:**
 - Select a video file for testing (e.g., 'sample_video.mp4').
 - Develop a Python script that uses Camshift for object tracking in the video.
 - Display the original video with the tracked object highlighted.
 - Provide user interactions for selecting the initial region of interest (ROI) to track.
 - Ensure that the script accurately tracks the object throughout the video.

Feature Extraction with Traditional Approaches

Pre lab:

Please read the solution below for understanding before starting the lab

<https://domino.ai/blog/feature-extraction-and-image-classification-using-deep-neural-networks>

1) Graded Lab Tasks (Allotted Time 1 Hour)

Lab Task: Feature Extraction with Traditional Approaches

- **Objective:**
 - Implement feature extraction using traditional approaches like SIFT, SURF, BRIEF, ORB, and HOG.
- **Instructions:**
 - Choose an image dataset for testing.
 - Develop a Python script that extracts features using SIFT, SURF, BRIEF, ORB, and HOG.
 - Display the original image along with key points or descriptors highlighted.
 - Compare the computational efficiency and accuracy of each feature extraction method.

Helping

materials:

<https://domino.ai/blog/feature-extraction-and-image-classification-using-deep-neural-networks>

Lab 09

OCR

Pre Lab:

Please read the solution below for understanding before starting the lab

<https://nanonets.com/blog/ocr-with-tesseract/>

<https://github.com/JaidedAI/EasyOCR>

Lab Task 1: Implementing PDF OCR with Tesseract in Python

- **Objective:**
 - Develop a Python script for implementing PDF OCR using the Tesseract engine, explaining the technology and incorporating OpenCV preprocessing.
- **Instructions:**
 - Select a PDF document containing text for testing.
 - Create a Python script that utilizes the Tesseract engine for OCR on the PDF.
 - Implement OpenCV preprocessing techniques to enhance OCR accuracy.

Lab Task 2: Implementing PDF OCR with EasyOCR in Python

- **Objective:**
 - Develop a Python script for implementing PDF OCR using the EasyOCR library, explaining the technology and incorporating any necessary preprocessing.
- **Instructions:**
 - Select a PDF document containing text for testing.
 - Create a Python script that utilizes the EasyOCR library for OCR on the PDF.
 - Implement any necessary preprocessing techniques to enhance OCR accuracy.
 - Provide a step-by-step guide/tutorial on how to implement PDF OCR using EasyOCR.
 - Highlight any specific features or capabilities of the EasyOCR library.

Helping

<https://nanonets.com/blog/ocr-with-tesseract/>
<https://github.com/JaidedAI/EasyOCR>

materials:

Lab 10

Identify Contours by Shape

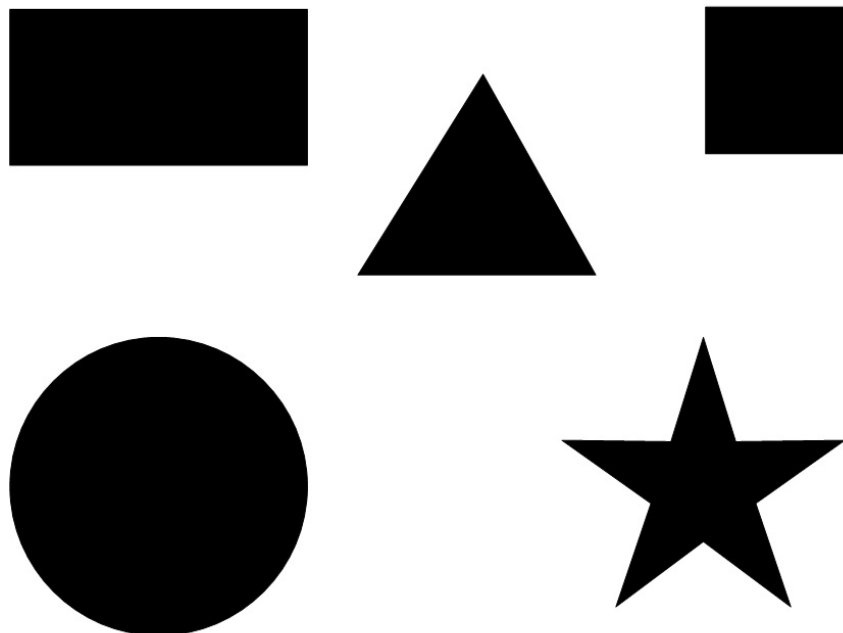
Lab Objective:

The objective of this mini-project is to develop a Python script that identifies and labels different shapes in an image based on their contours. The script will implement contour analysis and utilize shape properties to categorize and annotate each identified shape.

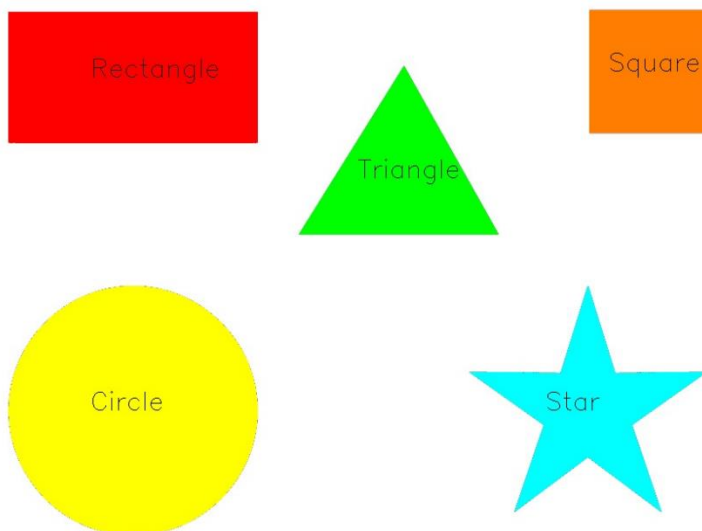
Instructions:

- **Image Loading and Preprocessing:**
 - Load the provided image (input image)
 - If needed, apply any preprocessing steps to enhance the accuracy of contour detection.
- **Contour Analysis:**
 - Use the `cv2.findContours` function to identify contours in the image.
 - Implement a loop to iterate through each contour.
- **Shape Identification:**
 - Implement a shape classification mechanism based on the number of sides of each contour.
 - Identify and label shapes such as triangles, squares, rectangles, stars, and circles.
- **Annotation and Visualization:**
 - Annotate each identified shape with a label.
 - Display the original image with the annotated shapes using Matplotlib.

Input Image



Final output



Lab 11

Blob Detection

Lab Objective:

In this mini-project, you will implement a Python script for blob detection using the OpenCV library. The script detects blobs in an image and visualizes the results by drawing red circles around the detected blobs. Follow the instructions below to complete the lab:

Instructions:

1. Library Import:

- Import the necessary libraries, including OpenCV and Matplotlib.

2. Image Loading and Preprocessing:

- Load the provided image (input image)
- Convert the image to grayscale using OpenCV.

3. Blob Detection:

- Set up the SimpleBlobDetector with default parameters.
- Use the detector to detect blobs in the grayscale image.

4. Visualization:

- Draw red circles around the detected blobs on a blank image using the cv2.drawKeypoints function.
- Display the original image and the image with annotated blobs side by side using Matplotlib.

Input Image



Final output



Lab 12

Counting Circles and Ellipses

Objective:

In this lab, you will develop a Python script to count circles and ellipses in an image using blob detection with the OpenCV library. Follow the instructions below to complete the lab:

Pre lab:

Before starting the lab, please read the following materials first.

<https://github.com/Meetkumar1/Counting-Circles-and-Ellipses-OpenCV/blob/master/Counting%20Circles%20and%20Ellipses%20OpenCV.ipynb>

Instructions:

- **Image Loading:**
 - Load the provided image (input image), or you can use an image of your choice containing circles and ellipses.
- **Default Blob Detection:**
 - Initialize the blob detector with default parameters using `cv2.SimpleBlobDetector_create()`.
 - Detect blobs in the image.
 - Draw the detected blobs as red circles on a blank image.
 - Display the original image and the image with annotated blobs.
- **Counting Total Blobs:**
 - Count and display the total number of blobs detected using the default parameters.
- **Filtered Blob Detection:**
 - Initialize a new set of parameters (`cv2.SimpleBlobDetector_Params()`).
 - Set filtering parameters to focus on circular blobs:
 - Area filtering (`params.filterByArea = True; params.minArea = 100`).
 - Circularity filtering (`params.filterByCircularity = True; params.minCircularity = 0.9`).
 - Convexity filtering (`params.filterByConvexity = False; params.minConvexity = 0.2`).
 - Inertia filtering (`params.filterByInertia = True; params.minInertiaRatio = 0.01`).
 - Create a new blob detector with the set parameters.
 - Detect blobs in the image using the new parameters.
 - Draw the circular blobs as green circles on a blank image.
 - Display the original image and the image with annotated circular blobs.
- **Counting Circular Blobs:**
 - Count and display the number of circular blobs detected with the filtered parameters.

Code sample

Blob count

```
# Load image
image = cv2.imread("../images/blobs.jpg", 0)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title('Original Image'); plt.show()

# Initialize the detector using the default parameters
detector = cv2.SimpleBlobDetector_create()

# Detect blobs
keypoints = detector.detect(image)

# Draw blobs on our image as red circles
blank = np.zeros((1,1))
blobs = cv2.drawKeypoints(image, keypoints, blank, (0,0,255),
                          cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

number_of_blobs = len(keypoints)
text = "Total Number of Blobs: " + str(len(keypoints))
cv2.putText(blobs, text, (20, 550), cv2.FONT_HERSHEY_SIMPLEX, 1, (100, 0, 255), 2)

row, col = 1, 2
fig, axs = plt.subplots(row, col, figsize=(15, 10))
fig.tight_layout()

axs[0].imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
axs[0].set_title('Original Image')
cv2.imwrite('input.jpg', image)

axs[1].imshow(cv2.cvtColor(blobs, cv2.COLOR_BGR2RGB))
axs[1].set_title('Blobs using default parameters')
cv2.imwrite('blobs.jpg', blobs)

plt.show()
```

Circles count

```

# Set our filtering parameters
# Initialize parameter setting using cv2.SimpleBlobDetector
params = cv2.SimpleBlobDetector_Params()

# Set Area filtering parameters
params.filterByArea = True; params.minArea = 100

# Set Circularity filtering parameters
params.filterByCircularity = True ; params.minCircularity = 0.9

# Set Convexity filtering parameters
params.filterByConvexity = False; params.minConvexity = 0.2

# Set inertia filtering parameters
params.filterByInertia = True; params.minInertiaRatio = 0.01

# Create a detector with the parameters
detector = cv2.SimpleBlobDetector_create(params)

# Detect blobs
keypoints = detector.detect(image)

```

```

# Draw blobs on our image as red circles
blank = np.zeros((1,1))
blobs = cv2.drawKeypoints(image, keypoints, blank, (0,255,0),
                          cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

number_of_blobs = len(keypoints)
text = "Number of Circular Blobs: " + str(len(keypoints))
cv2.putText(blobs, text, (20, 550), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 100, 255), 2)

# Show blobs
row, col = 1, 2
fig, axs = plt.subplots(row, col, figsize=(15, 10))
fig.tight_layout()

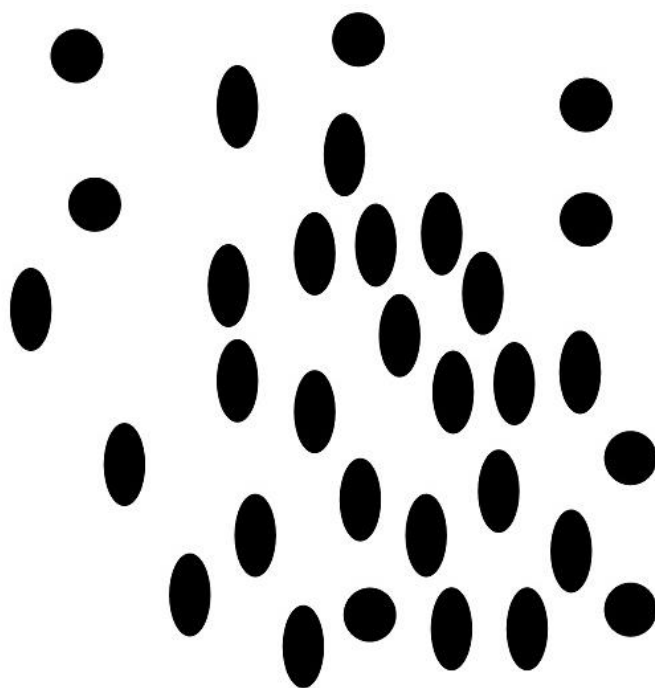
axs[0].imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
axs[0].set_title('Original Image')
cv2.imwrite('input.jpg', image)

axs[1].imshow(cv2.cvtColor(blobs, cv2.COLOR_BGR2RGB))
axs[1].set_title('Filtering Circular Blobs Only')
cv2.imwrite('circle_blobs.jpg', blobs)

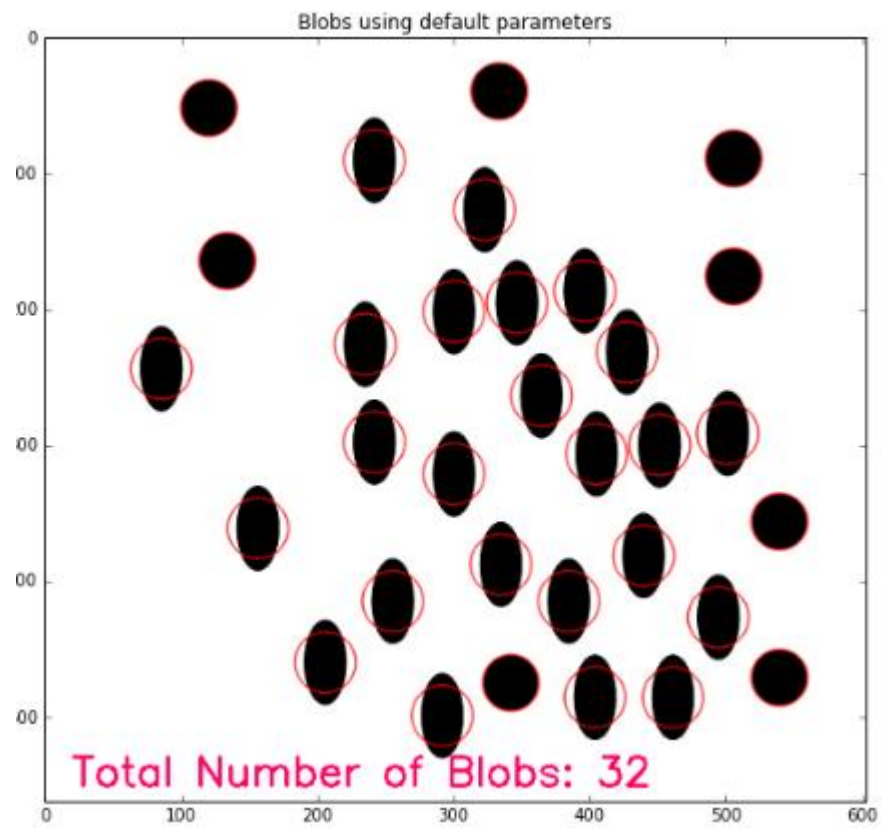
plt.show()

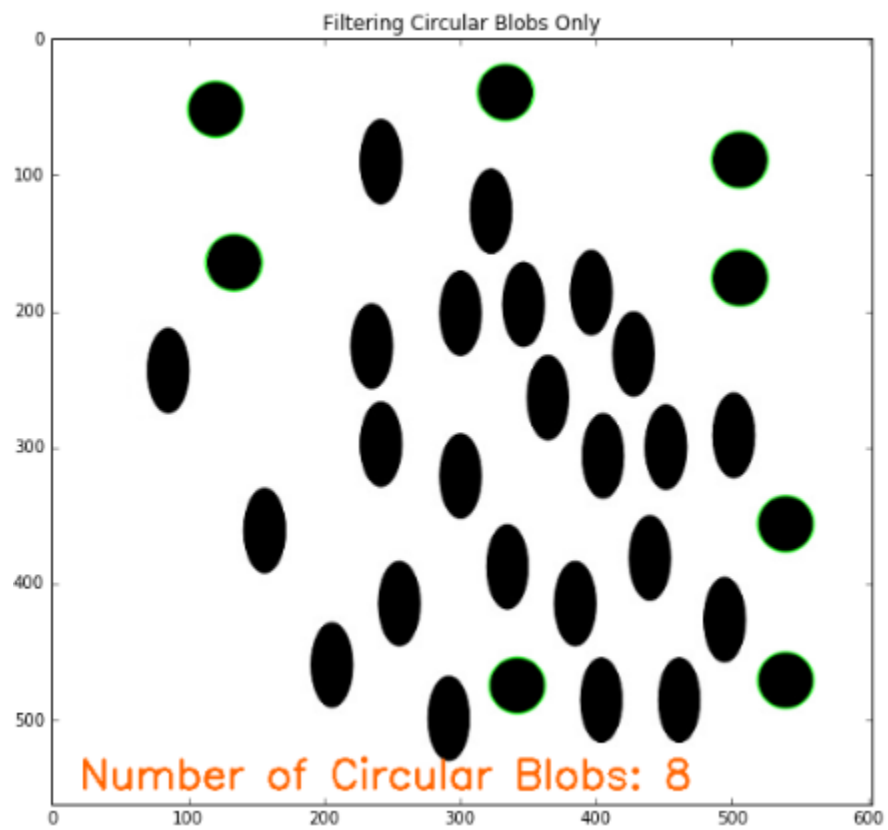
```

Input images



Final output image





Lab 13

Face Recognition – Unlock Your Computer With Your Face!

Pre lab:

Before starting the project, please read the following materials first.
https://github.com/rchavezj/OpenCV_Projects/blob/master/Sec07_Simple_Machine_Learning/03_Face_Recognition_%C3%B1_Unlock_Your_Computer_With_Your_Face/%2003_Face_Recognition_%C3%B1_Unlock_Your_Computer_With_Your_Face.ipynb

Objective:

In this lab, you will create a facial recognition system to unlock a computer using your face. The lab is divided into three steps: creating training data, training the model, and running the facial recognition system.

Step 1: Create Training Data

- **Objective:**
 - Capture 100 samples of your face using the webcam to create training data.
- **Instructions:**
 - Load the Haar cascade face classifier and define a function (**face_extractor**) to detect and crop faces from the webcam feed.
 - Initialize the webcam and collect 100 samples of your face.
 - Save each face sample with a unique name in the specified directory.
- **Grading Criteria:**
 - Successful collection of 100 face samples.
 - Proper saving of face samples with unique names.
 - Display live count of collected face samples.

Step 2: Train Model

- **Objective:**
 - Train a facial recognition model using the collected face samples.
- **Instructions:**
 - Load the training data from the directory where face samples are saved.
 - Create arrays for training data and labels.
 - Train the LBPH facial recognizer using OpenCV with the collected training data.
- **Grading Criteria:**
 - Successful loading of training data and labels.
 - Proper training of the LBPH facial recognizer.
 - Display a message indicating successful training.

Step 3: Run Facial Recognition

- **Objective:**
 - Implement a facial recognition system that unlocks the computer based on the trained model.
- **Instructions:**
 - Define a function (**face_detector**) to detect faces in the webcam feed.
 - Open the webcam and continuously detect and recognize faces.
 - Display the webcam feed with rectangles around detected faces and unlock/lock status based on confidence levels.
- **Grading Criteria:**
 - Successful detection and recognition of faces in the webcam feed.
 - Display of confidence levels and unlock/lock status.
 - Proper handling of scenarios where no face is found.

Lab 14

Training YOLOv8 on a Custom Dataset

Pre lab:

Before starting the project, please read the following materials first.

<https://learnopencv.com/train-yolov8-on-custom-dataset/>

<https://blog.paperspace.com/train-yolov5-custom-data/>

<https://www.analyticsvidhya.com/blog/2022/08/how-to-train-a-custom-object-detection-model-with-yolov7/>

<https://machinelearningprojects.net/train-yolov7-on-the-custom-dataset/>

Objective:

In this lab exercise, you will demonstrate your ability to set up and train YOLOv8 on a custom dataset for object detection. The exam consists of multiple steps, including cloning the YOLOv8 repository, installing necessary dependencies, preparing a custom dataset, configuring YOLOv8 for training, and initiating the training process. You will use an open-source dataset of your choice for this exercise.

Instructions:

Step 1: Clone YOLOv8 Repository

Step 2: Install Dependencies

- **Objective:**
 - Install the required packages for YOLOv8.

Step 3: Prepare Custom Dataset

- **Objective:**
 - Prepare a custom dataset for training using an open-source dataset of your choice.
- **Instructions:**
 - Organize your dataset with labeled images and corresponding annotations.
 - Ensure the dataset follows the required YOLO format.

Step 4: Configure YOLOv8 for Training (10 points)

- **Objective:**
 - Configure YOLOv8 for training with the custom dataset.
- **Instructions:**
 - Update the YOLOv8 configuration file (yolov8/data/custom.yaml) with the details of your custom dataset.

Step 5: Start Training

- **Objective:**
 - Initiate the training process with YOLOv8.

Step 6: Evaluation

-
- **Objective:**
 - Evaluate the performance of the trained YOLOv8 model using videos or images.